

Un premier programme d'apprentissage en Python

Bibliothèques à installer

- NumPy (pip install numpy)
- Matplotlib (pip install matplotlib)
- Apprentissage (pip install scikit-learn)

Le problème à résoudre

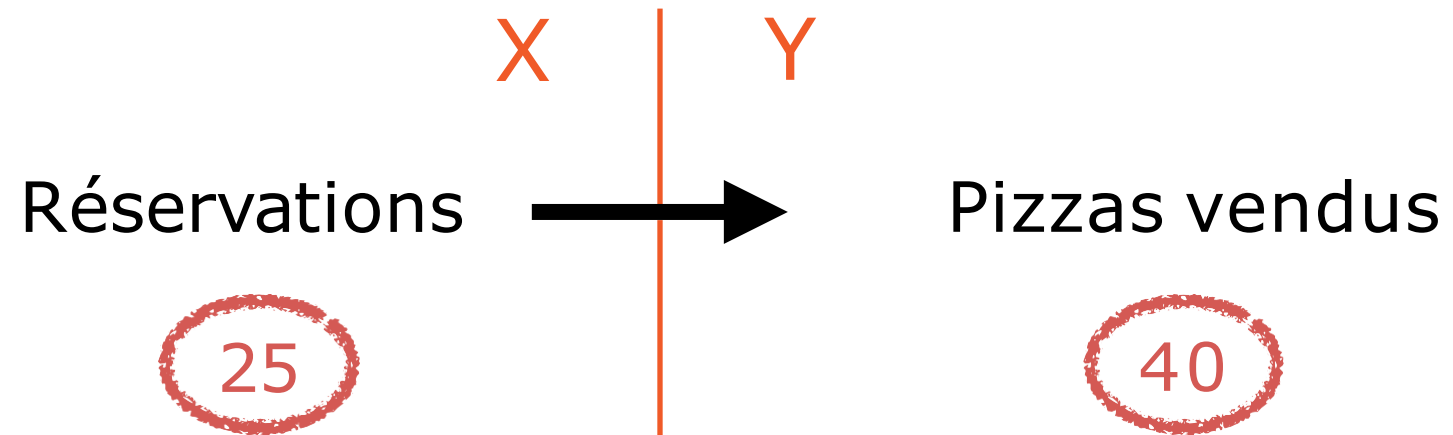


Nombre de réservations



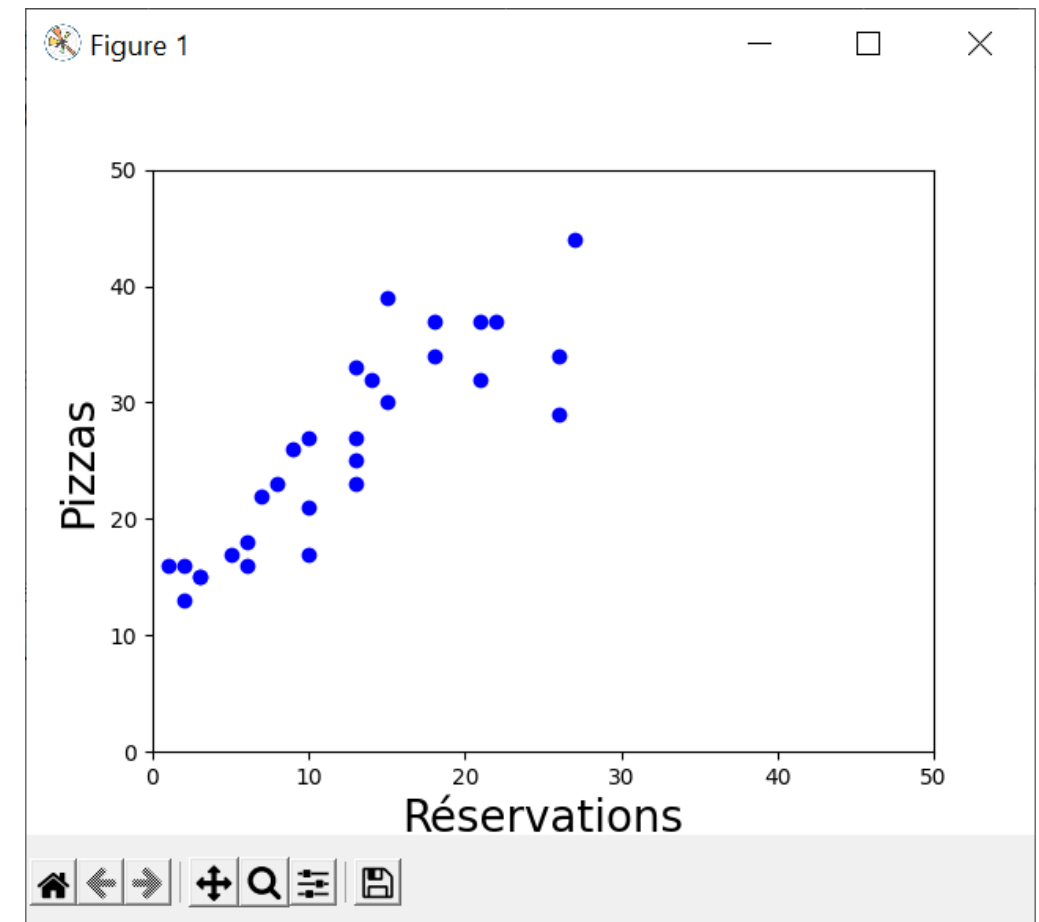
Pizzas vendus

Nous voulons trouver la relation entre...

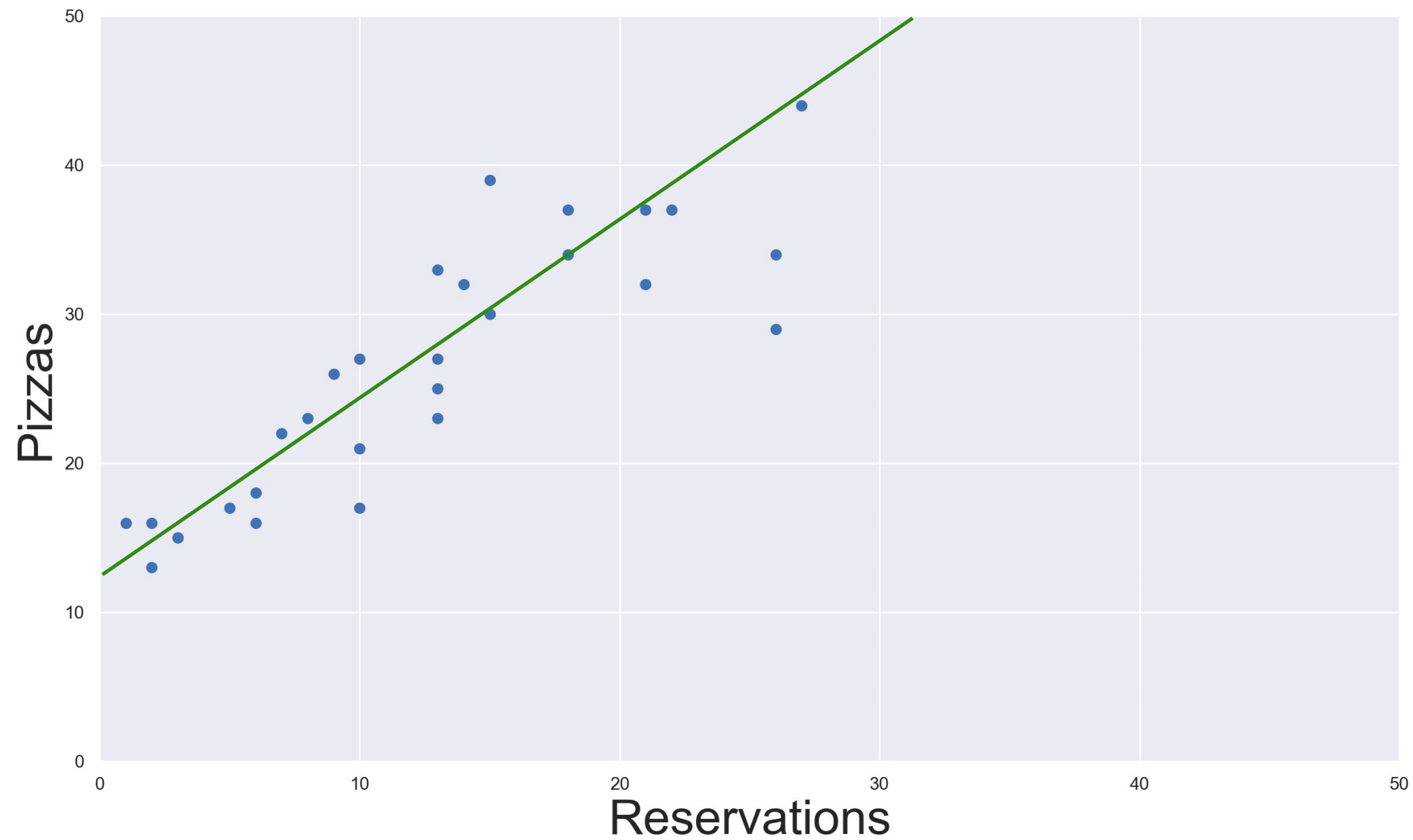


Fichier de données: pizza.txt

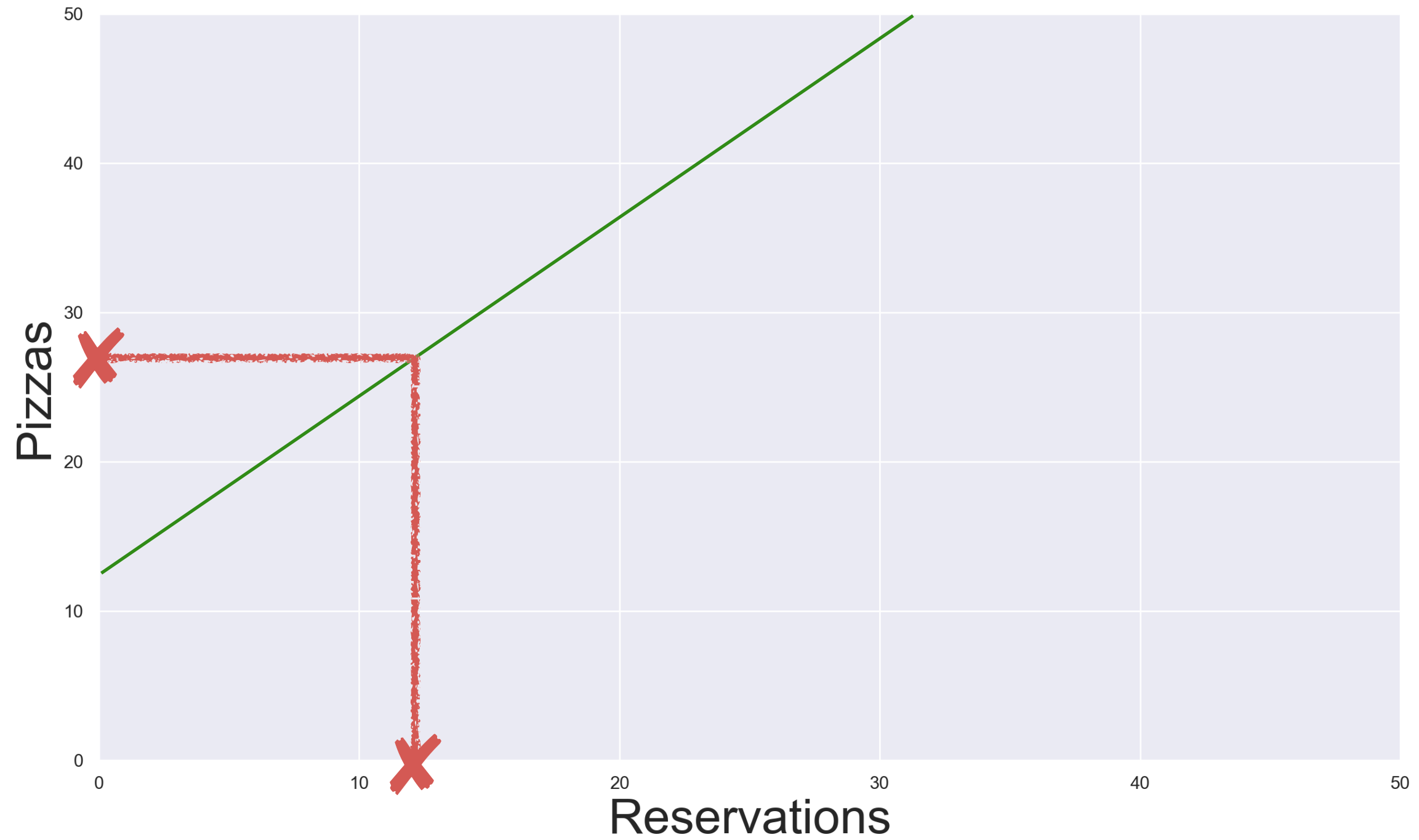
	Réservations	Pizzas
1		
2	13	33
3	2	16
4	14	32
5	23	51
6	13	27
7	13	25
8	1	16
9	18	34



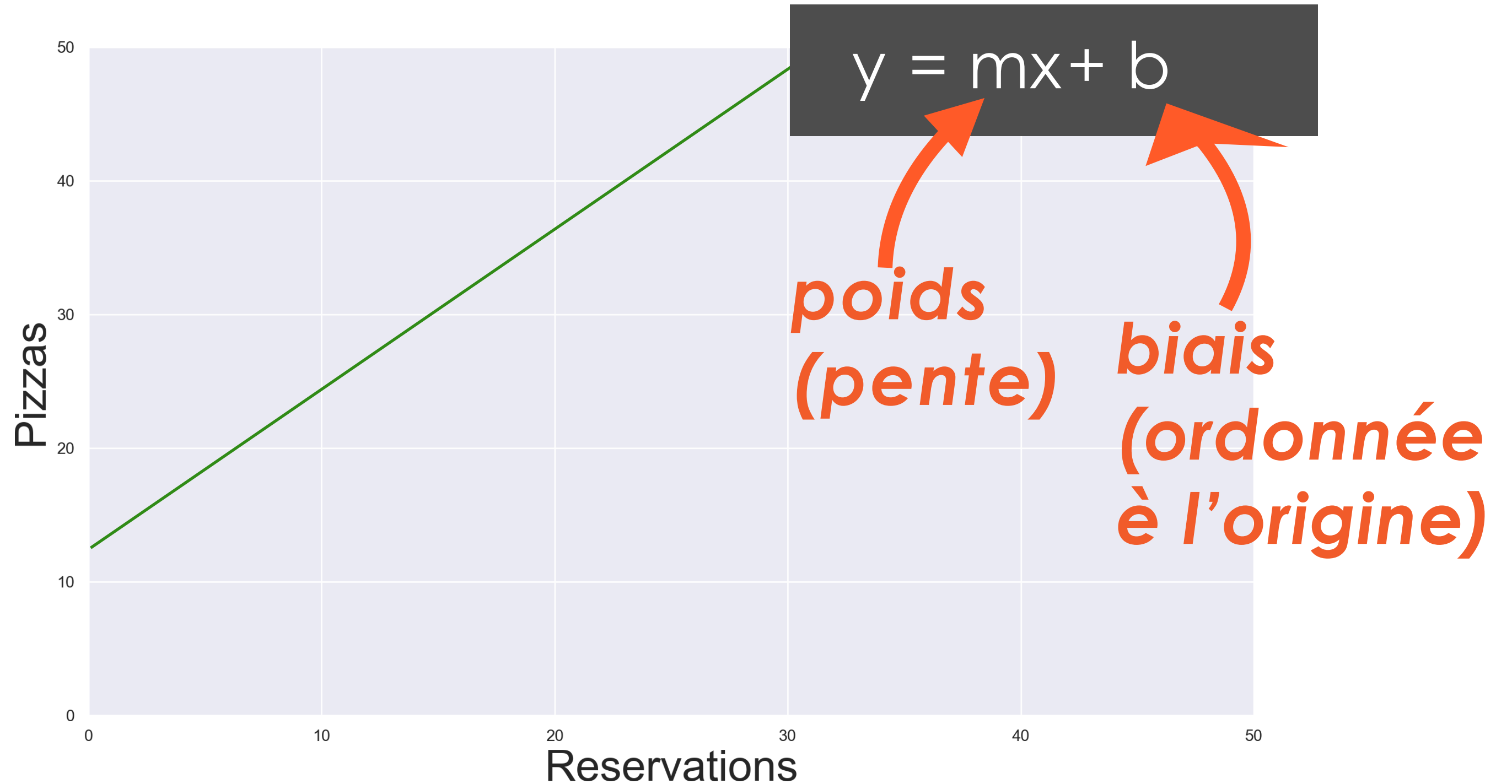
Approximer les données



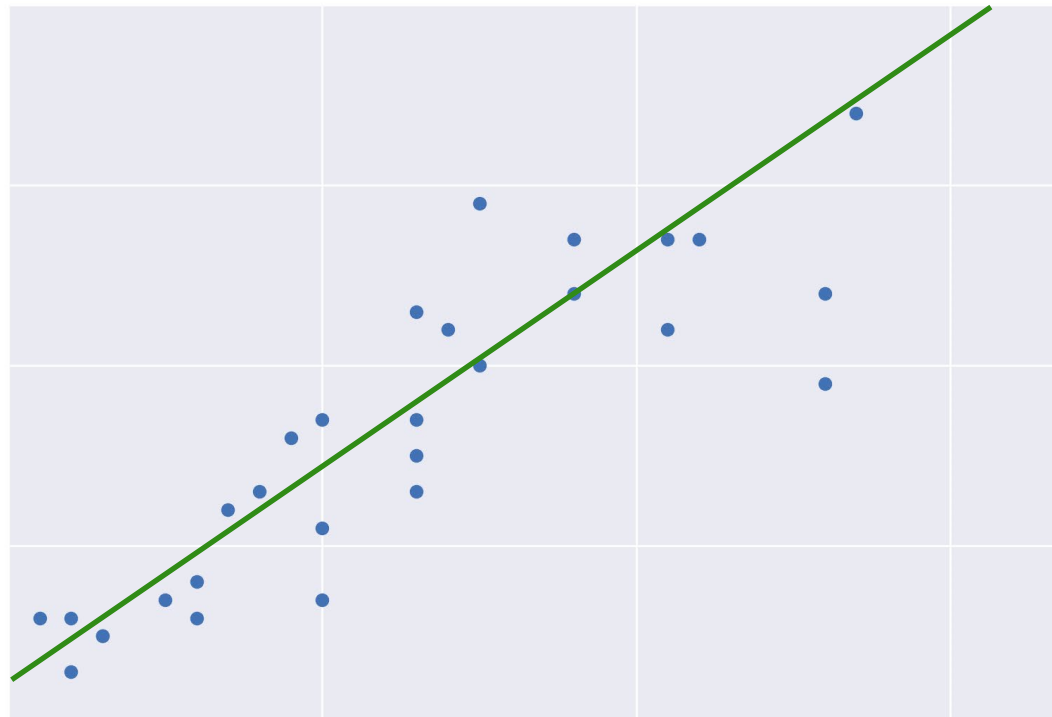
Régression linéaire



L'équation d'une droite

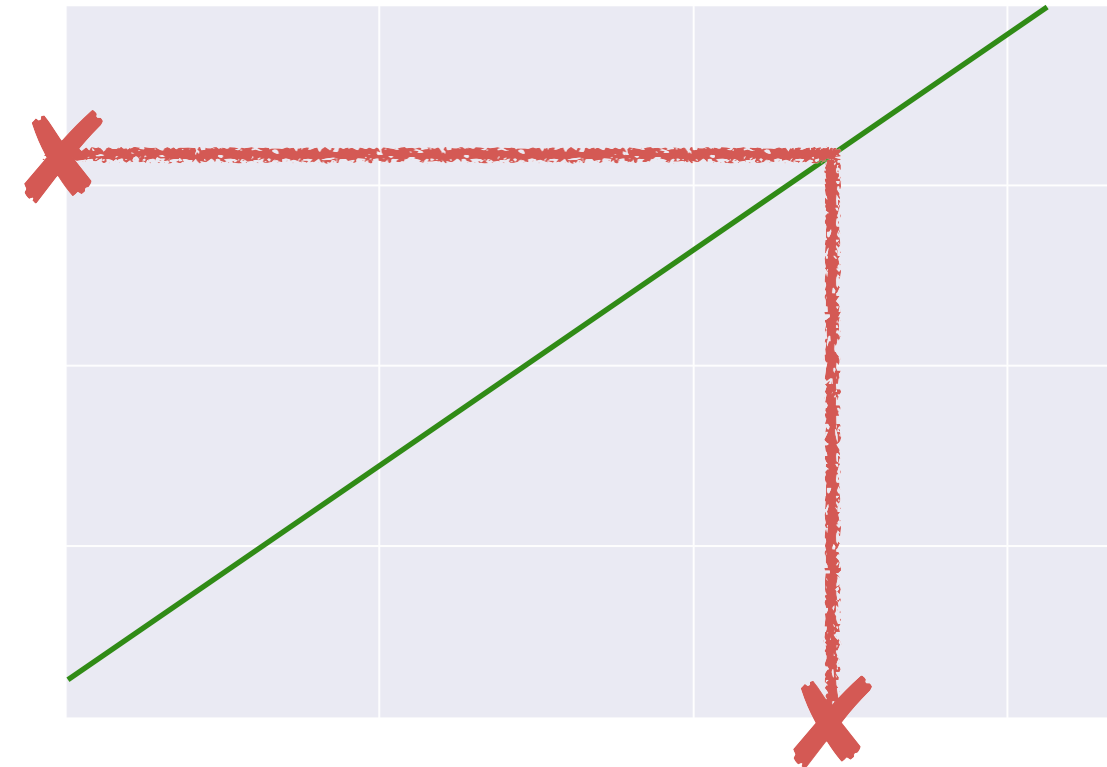


Rappel: les deux phases en AAS



Phase d'entraînement

Approximer les données étiquetées par un modèle



Phase de prédiction

Utiliser un modèle pour étiqueter des données nouvelles

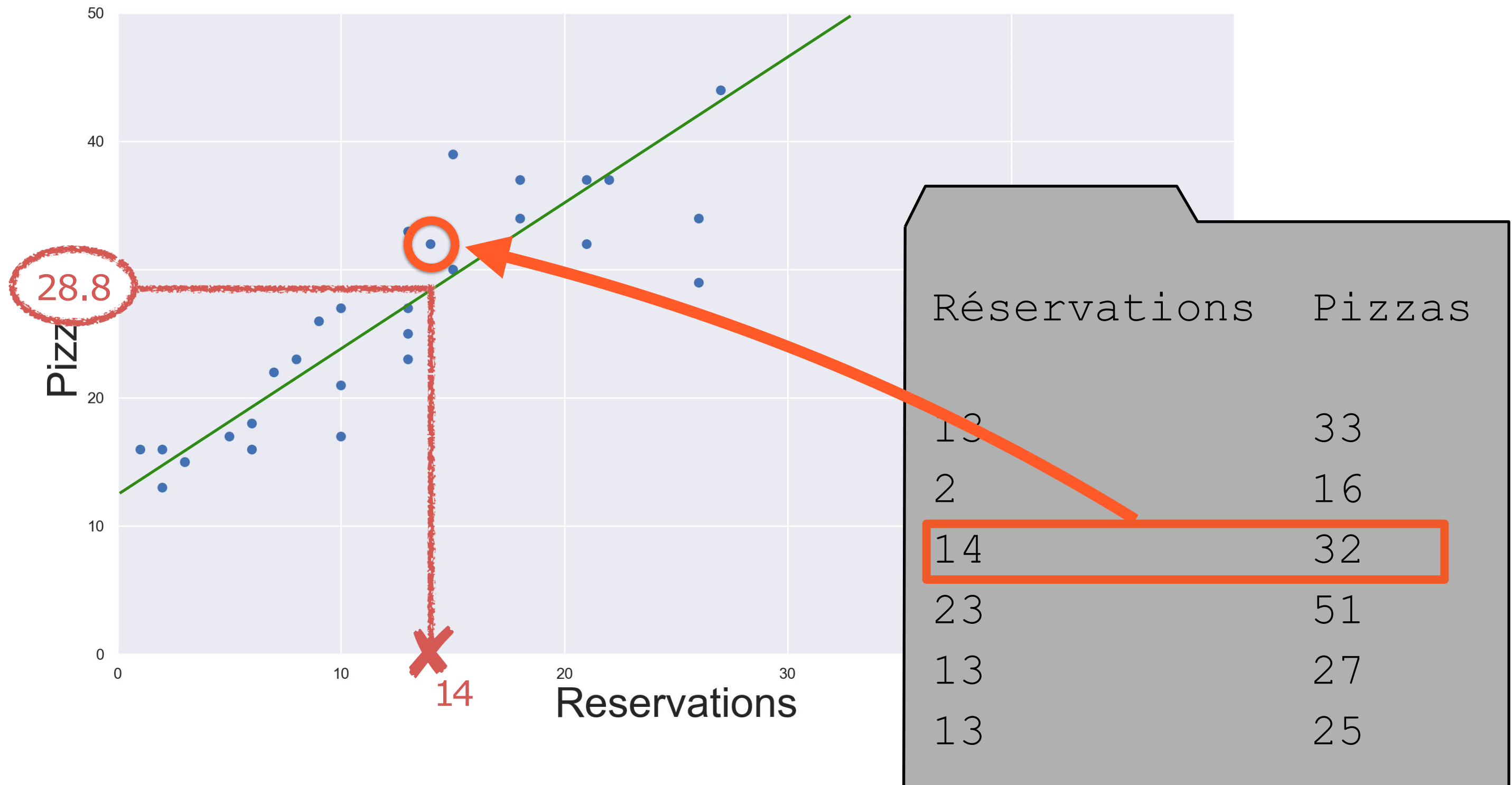
La phase d'entraînement

Le but est de trouver une droite qui minimisera les erreurs de prédiction.

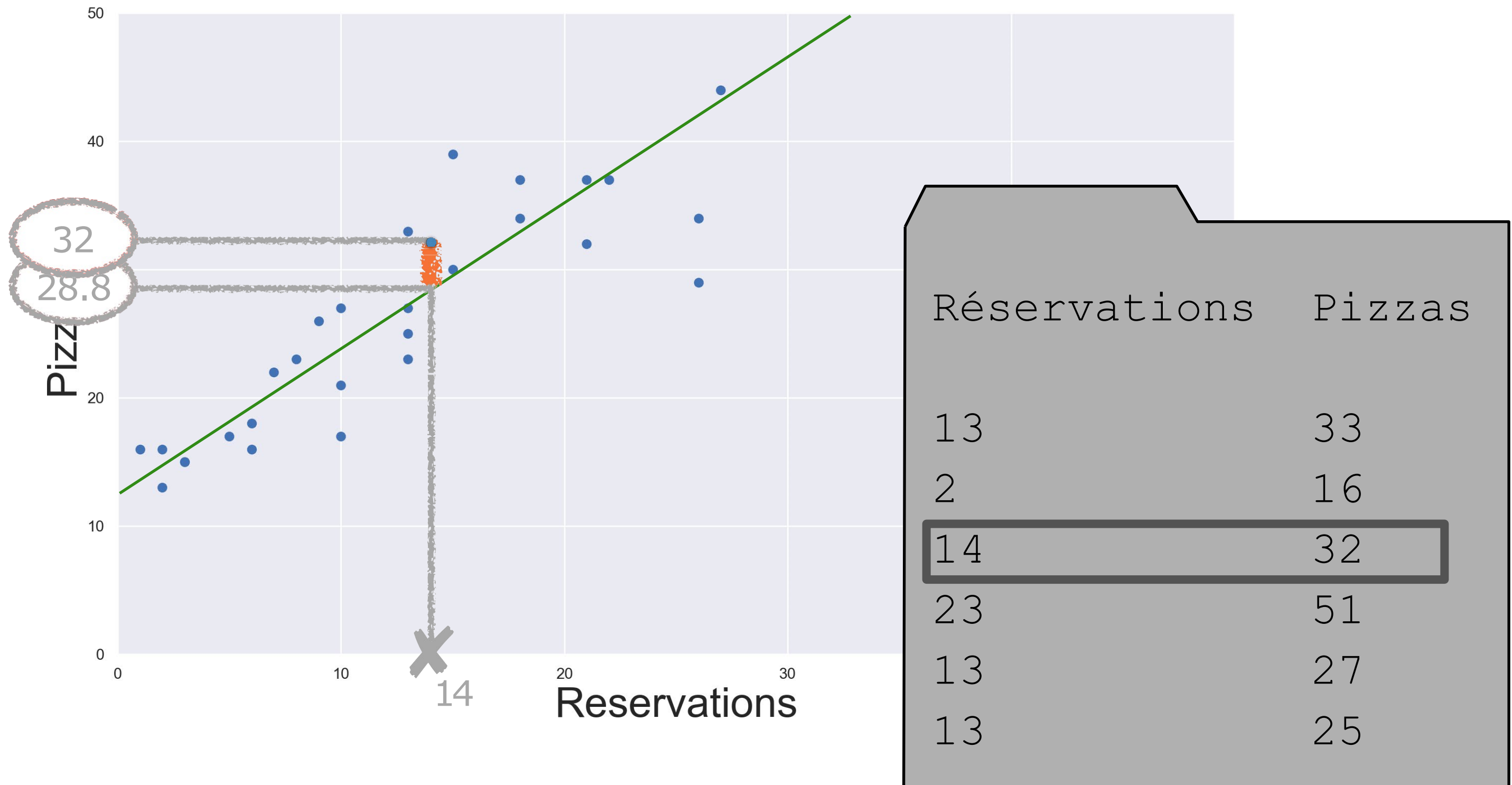


On essaie d'ajuster la meilleure ligne afin de minimiser les erreurs de prédictions pour les valeurs de y étant donné une valeur de x . Les erreurs sont généralement calculées en tant que RMSE (erreur quadratique moyenne) des distances entre les données et la ligne que vous essayez d'ajuster.

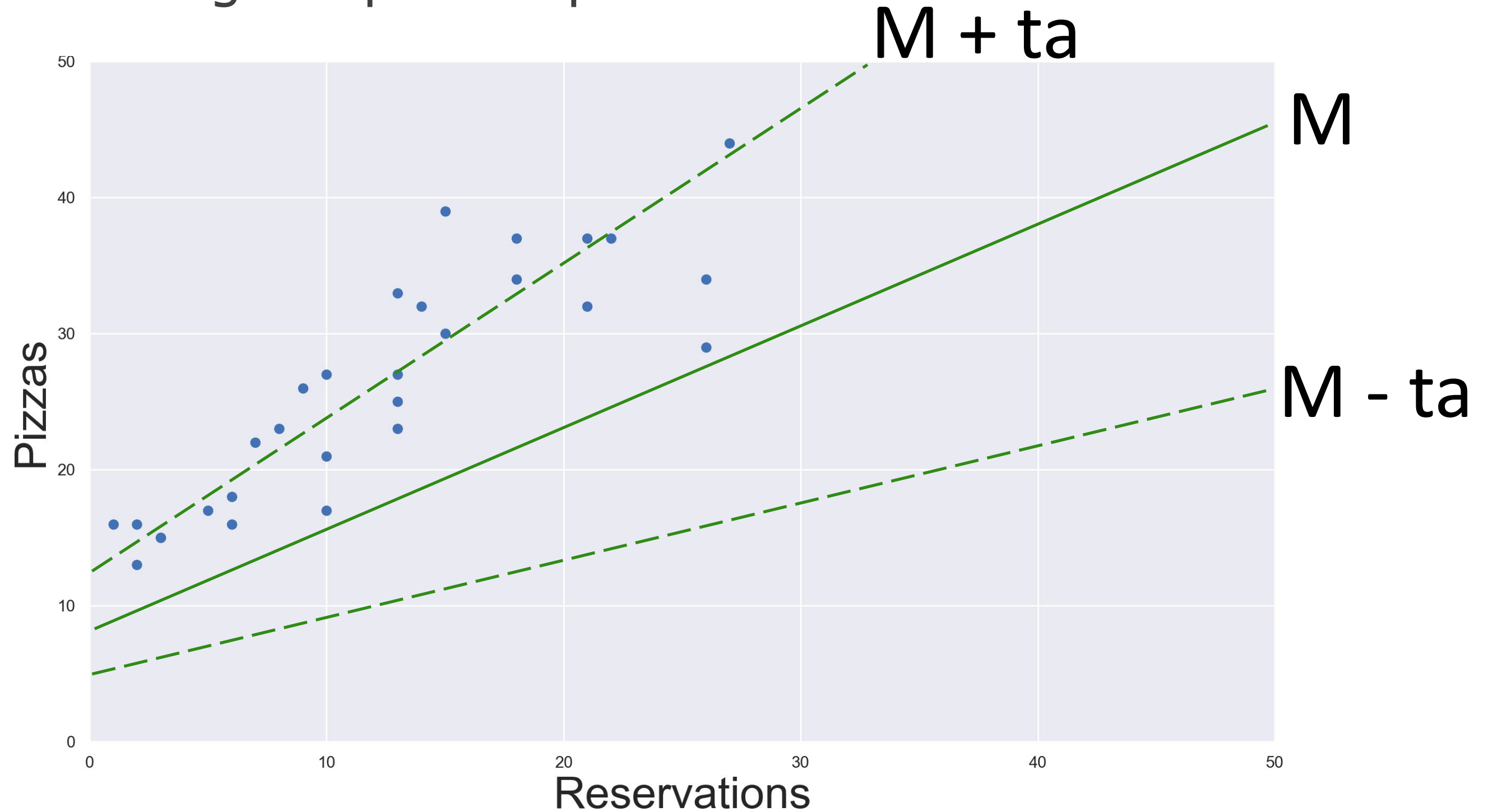
Chaque ligne est imparfaite



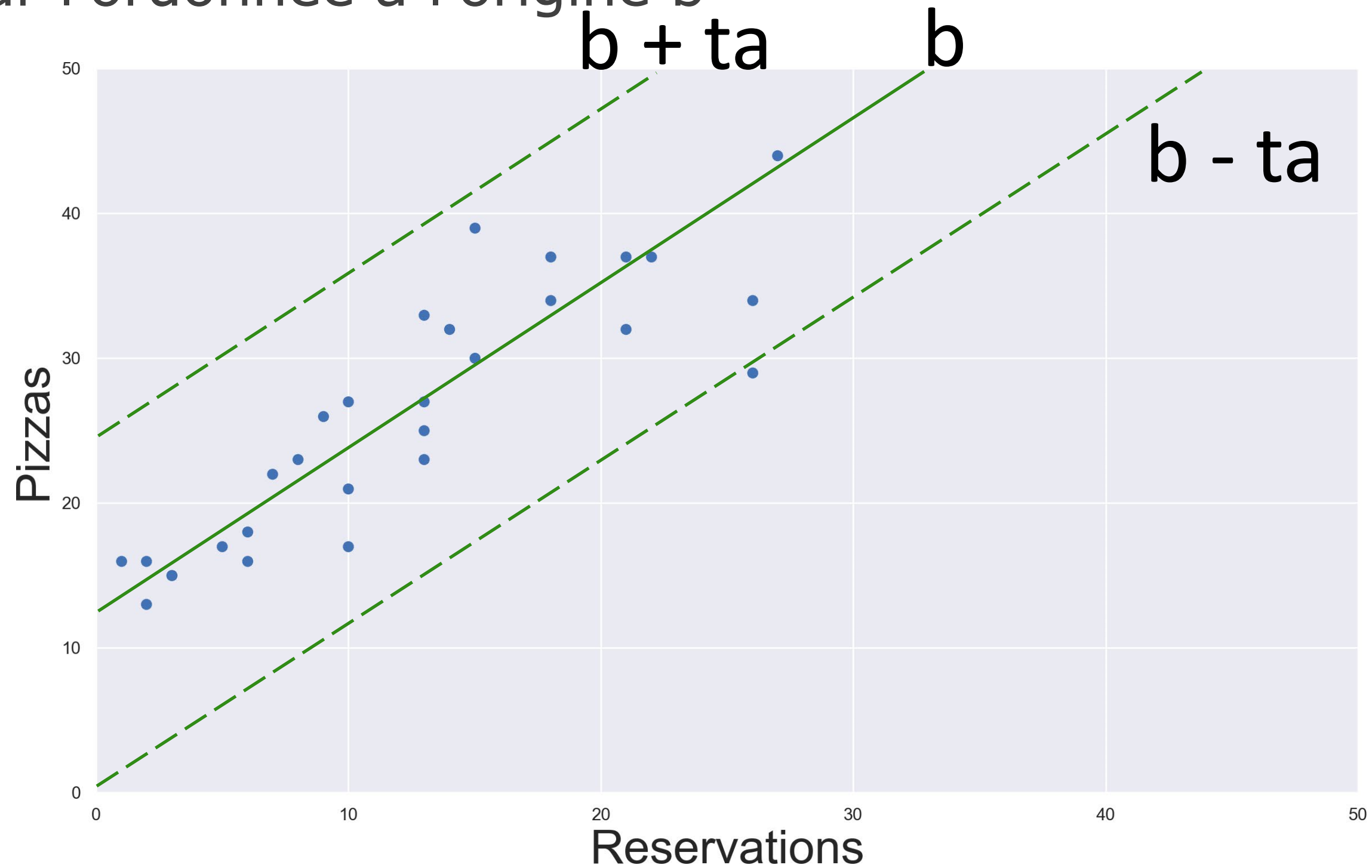
Presque chaque ligne a une erreur



L'algorithme d'entraînement implique un taux d'apprentissage α pour la pente



L'algorithme d'entraînement implique un taux d'apprentissage η pour l'ordonnée à l'origine b



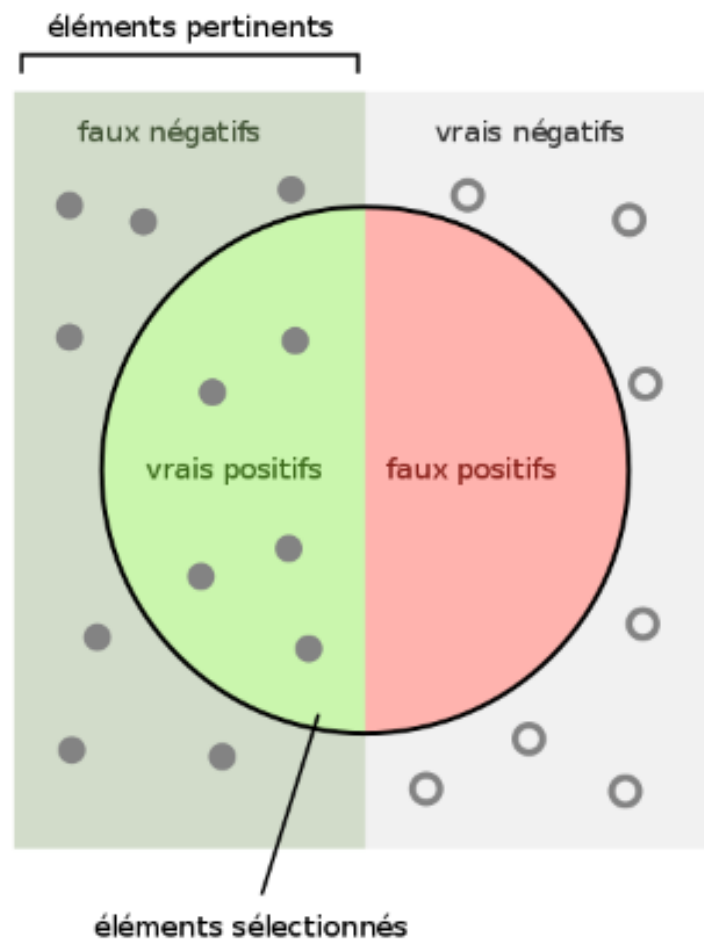
Code Python à compléter

```
1 import numpy as np, sys
2 import matplotlib.pyplot as plt
3 def tracerDonnees(X,Y):
4     plt.xlabel("Réservations", fontsize = 20) # abscisse
5     plt.ylabel("Pizzas vendues", fontsize = 20) # ordonné
6     plt.scatter(X, Y, label = 'Données')
7 # Prédiction de Y à partir des X pour un modèle (m,b)
8 def predire(X, m, b):
9     pass
10 # Erreur (perte) moyenne pour un modèle (m,b)
11 def perte(X, Y, m, b): # moyenne des pertes entre un modèle (m,b) et la réalité Y
12     pass
13 # Entraînement pour trouver un modèle (m,b)
14 def entraine(X, Y, iterations, ta): # ta = taux d'apprentissage
15     # iterations: pour éviter les vas-et-vient infinis
16     # but: trouver m et b
17     pass
18 if __name__ == "__main__":
19     # Importer les données dans deux tableaux distincts (unpack)
20     X, Y = np.loadtxt("pizza.txt", skiprows=1, unpack=True)
21     tracerDonnees(X,Y)
22     plt.show()
23     # On entraîne de notre modèle
24     m, b = entraine(X, Y, 2000, 0.01)
25     # Visualisation et prédiction avec notre modèle
26     print("Notre modèle: m = " + str(round(m, 2)) + " b = " + str(round(b, 2)))
27     tracerDonnees(X, Y)
28     # [X1, X2] [Y1, Y2] la droite que nous avons trouvée
29     plt.plot([0, 30], [predire(0, m, b), predire(30, m, b)], color = "r", label='Droite apprise')
30     plt.legend(loc = 'upper left')
31     plt.show()
32     # Faisons une prédiction avec notre modèle
33     reservations = 21
34     print("Prédictions: avec " + str(reservations)
35           + " réservations, on prépare " + str(int(predire(reservations, m, b))) + " pizzas.")
```

Notre modèle avec 2000 itérations:



Métriques pour évaluer une classification



Compromis entre precision et rappel:
$$F = 2 \cdot \frac{(\text{précision} \cdot \text{rappel})}{(\text{précision} + \text{rappel})}$$

Exactitude = $(VP + VN) / (VP + VN + FP + FN)$

